

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА
ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление 02.03.01 Математика и компьютерные науки

Отчёт по дисциплине «Математическая логика и теория автоматов»

Лабораторная работа №4 «Расширение языка
Милан»

Вариант: 18

Студент: _____

Черданцев Артём Сергеевич

Преподаватель: _____

Востров Алексей Владимирович

«____» _____ 20__ г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Математическое описание	4
2 Особенности реализации	8
2.1 Лексический анализ	8
2.2 Синтаксический анализ и генерация кода стековой машины	9
2.2.1 Логические операторы	9
2.2.2 Побитовые операции	10
2.2.3 Изменения в обработке условий и выражений	11
2.2.4 Модификация синтаксического анализа выражений	11
3 Результаты работы программы	13
Заключение	16
Список источников	17

Введение

Цель данной работы — расширить компилятор языка Милан, реализованный на языке C++.

Задачи:

- Реализовать поддержку булевых констант true и false.
- Реализовать поддержку конъюнкции (&&)
- Реализовать поддержку дизъюнкции (||).
- Реализовать поддержку Штриха Шеффера (!& - Отрицание конъюнкции)
- Реализовать поддержку отрицания (!).
- Реализовать поддержку побитовой дизъюнкции (|).
- Реализовать поддержку побитовой конъюнкции (&).

С точки зрения вычисления выражений необходимо учесть особенности операций || и && по сравнению с операциями | и & соответственно (для || если первый операнд истина, то второй не вычисляется, для && если первый операнд ложь, то второй не вычисляется).

1 Математическое описание

Компилятор SMilan состоит из следующих компонентов:

- лексический анализатор;
- синтаксический анализатор;
- генератор кода виртуальной машины.

Лексический анализатор распознает код программы и разбивает ее лексемы на отдельные фрагменты (Token). Также лексический анализатор удаляет пробельные символы и комментарии.

Синтаксический анализатор читает сформированную лексическим анализатором последовательность лексем и проверяет ее соответствие грамматике языка Милан. Проверка осуществляется методом рекурсивного спуска. В процессе анализа программы синтаксический анализатор генерирует набор команд для виртуальной машины.

Грамматика языка Милан распознается нисходящим парсером. У нисходящего парсера продукции грамматики раскрываются с начального нетерминала, до получения требуемой последовательности токенов.

При восстановлении синтаксического дерева при нисходящем разборе слева направо последовательно анализирует все поддеревья, принадлежащие самому левому нетерминалу. Когда самым левым становится другой нетерминал, анализируется уже он. При этом проверяются все правила, выбор которых для текущего нетерминала определяется на основе анализа очередного терминала входной цепочки.

Грамматика является контекстно-свободной, так как левые части всех продукций являются одиночными нетерминалами, а значит есть возможность применить продукцию к нетерминалу независимо от контекста этого нетерминала.

Грамматика языка Милан является LL(1) грамматикой, так как необходимо просмотреть поток всего на один символ вперед при принятии решения о том, какое правило грамматики необходимо применить.

Исходная Грамматика языка Милан в расширенной форме Бэкуса-Наура приведена ниже.

```
<program> ::= 'begin' <statementList> 'end'
<statementList> ::= <statement> ';' <statementList> | ε
<statement> ::= <ident> ':' <expression>
| 'if' <relation> 'then' <statementList> ['else' <statementList>] 'fi'
| 'while' <relation> 'do' <statementList> 'od'
| 'write' '(' <expression> ')'

<expression> ::= <term> {<addop> <term>}
<term> ::= <factor> {<mulop> <factor>}
<factor> ::= <ident> | <number> | '(' <expression> ')'
<relation> ::= <expression> <cmp> <expression>
<addop> ::= '+' | '-'
```

$\langle \text{multop} \rangle ::= '*' \mid '/'$
 $\langle \text{cmp} \rangle ::= '=' \mid '!=' \mid '<' \mid '<=' \mid '>' \mid '>='$
 $\langle \text{ident} \rangle ::= \langle \text{letter} \rangle \{ \langle \text{letter} \rangle \mid \langle \text{digit} \rangle \}$
 $\langle \text{letter} \rangle ::= 'a' \mid 'b' \mid 'c' \mid \dots \mid 'z' \mid 'A' \mid 'B' \mid 'C' \mid \dots \mid 'Z'$
 $\langle \text{digit} \rangle ::= '0' \mid '1' \mid '2' \mid '3' \mid '4' \mid '5' \mid '6' \mid '7' \mid '8' \mid '9'$

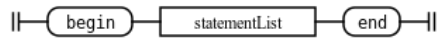
Измененная грамматика языка Милан согласно моему варианту приведена ниже. Красным выделены изменённые фрагменты.

$\langle \text{program} \rangle ::= \text{'begin'} \langle \text{statementList} \rangle \text{'end'}$
 $\langle \text{statementList} \rangle ::= \langle \text{statement} \rangle ';' \langle \text{statementList} \rangle \mid \varepsilon$
 $\langle \text{statement} \rangle ::= \langle \text{ident} \rangle ':' \langle \text{expression} \rangle$
 $\mid \text{'if'} \langle \text{relation} \rangle \text{'then'} \langle \text{statementList} \rangle [\text{'else'} \langle \text{statementList} \rangle] \text{'fi'}$
 $\mid \text{'while'} \langle \text{relation} \rangle \text{'do'} \langle \text{statementList} \rangle \text{'od'}$
 $\mid \text{'write'} \text{'('} \langle \text{expression} \rangle \text{'})'}$

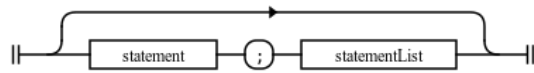
$\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ \langle \text{addop} \rangle \langle \text{term} \rangle \}$
 $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ \langle \text{mulop} \rangle \langle \text{factor} \rangle \}$
 $\langle \text{factor} \rangle ::= \langle \text{ident} \rangle \mid \langle \text{number} \rangle \mid \text{'('}, \langle \text{logicalOrExpr} \rangle, \text{'})' \mid \text{'-'}, \text{factor} \mid \text{'!'}, \text{factor} \mid \text{'read'}$;
 $\langle \text{relation} \rangle ::= \text{expression}, [\text{cmp}, \text{expression}]$;
 $\langle \text{equalityExpr} \rangle ::= \langle \text{relation} \rangle, [\text{eq}, \langle \text{relation} \rangle]$;
 $\langle \text{bitwiseAndExpr} \rangle ::= \langle \text{equalityExpr} \rangle, \text{'\&'} \langle \text{equalityExpr} \rangle$;
 $\langle \text{bitwiseOrExpr} \rangle ::= \langle \text{bitwiseAndExpr} \rangle, \langle \text{bitwiseAndExpr} \rangle$;
 $\langle \text{logicalAndExpr} \rangle ::= \langle \text{bitwiseOrExpr} \rangle, \text{'\&\&'} \langle \text{bitwiseOrExpr} \rangle$;
 $\langle \text{logicalOrExpr} \rangle ::= \langle \text{logicalAndExpr} \rangle, \mid \langle \text{logicalAndExpr} \rangle$;
 $\langle \text{unaryop} \rangle ::= \text{'!'}$
 $\langle \text{addop} \rangle ::= \text{'+'} \mid \text{'-'}$
 $\langle \text{multop} \rangle ::= '*' \mid '/'$
 $\langle \text{eq} \rangle ::= \text{'='} \mid \text{'!='}$
 $\langle \text{cmp} \rangle ::= '<' \mid '<=' \mid '>' \mid '>='$
 $\langle \text{ident} \rangle ::= \langle \text{letter} \rangle \{ \langle \text{letter} \rangle \mid \langle \text{digit} \rangle \}$
 $\langle \text{letter} \rangle ::= 'a' \mid 'b' \mid 'c' \mid \dots \mid 'z' \mid 'A' \mid 'B' \mid 'C' \mid \dots \mid 'Z'$
 $\langle \text{digit} \rangle ::= '0' \mid '1' \mid '2' \mid '3' \mid '4' \mid '5' \mid '6' \mid '7' \mid '8' \mid '9'$

На рис. 1-2 представлены синтаксические диаграммы языка Милан, соответствующие моей измененной грамматике.

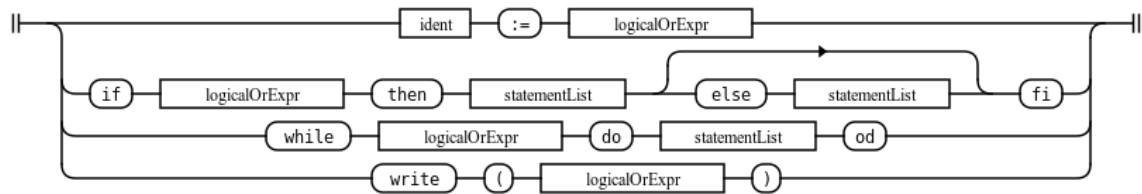
program:



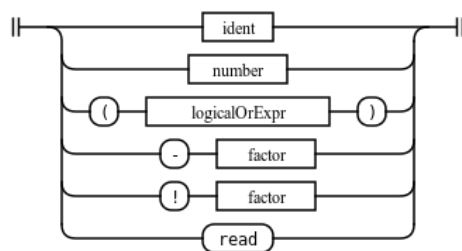
statementList:



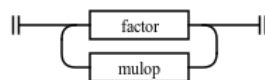
statement:



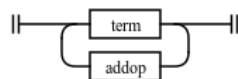
factor:



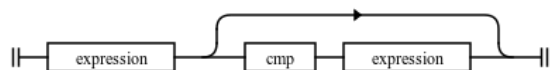
term:



expression:



relation:



equalityExpr:



bitwiseAndExpr:



bitwiseOrExpr:



Рис. 1: Синтаксическая диаграмма языка Милан ч.1

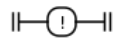
logicalAndExpr:



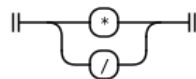
logicalOrExpr:



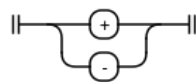
unaryop:



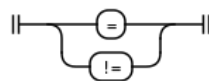
mulop:



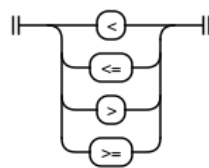
addop:



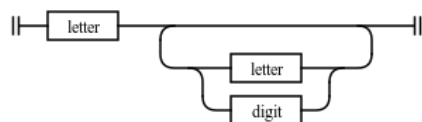
eq:



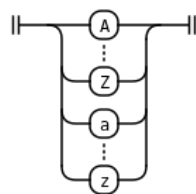
cmp:



ident:



letter:



digit:

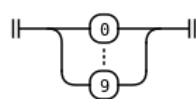


Рис. 2: Синтаксическая диаграмма языка Милан ч.2

2 Особенности реализации

2.1 Лексический анализ

В модуле scanner (файлы scanner.h и scanner.cpp) определен лексический анализатор языка Милан. Были внесены изменения для добавления новых операций.

1. Добавлены новые лексемы для операций побитового и логического "И" "ИЛИ" и "НЕ" а также добавлены лексемы для булевых значений:

```
1 enum Token {
2     T_FALSE,           // Ключевое слово "false"
3     T_TRUE,            // Ключевое слово "true"
4     T_BITWISEANDOP,    // Лексема для "&" (побитовое И)
5     T_BITWISEOROP,     // Лексема для "|" (побитовое ИЛИ)
6     T_LOGICALANDOP,    // Лексема для "&&" (логическое И)
7     T_LOGICALOROP,     // Лексема для "||" (логическое ИЛИ)
8     T_LOGICALNOTOP,    // Лексема для "!" (логическое НЕ)
9 };
```

2. Обновлено строковое представление лексем:

```
1 static const char *tokenNames_[] = {
2     "false",
3     "true",
4     "&",
5     "|",
6     "&&",
7     "||",
8     "!",
9 };
```

3. В основной функции nextToken добавлена обработка новых лексем:

Для символов &, |, и ! реализована проверка на логические операторы "&&" и "||" а также на побитовые операции:

```
1 case '&':
2     nextChar();
3     if (ch_ == '&') {
4         nextChar();
5         token_ = T_LOGICALANDOP;
6     } else {
7         token_ = T_BITWISEANDOP;
8     }
9     break;
10
11 case '|':
12     nextChar();
13     if (ch_ == '|') {
14         nextChar();
```



```

15         token_ = T_LOGICALOROP;
16     } else {
17         token_ = T_BITWISEOROP;
18     }
19     break;
20
21 case '!':
22     nextChar();
23     if (ch_ == '=') {
24         nextChar();
25         token_ = T_CMP;
26         cmpValue_ = C_NE;
27     } else {
28         token_ = T_LOGICALNOTOP;
29     }
30     break;
31

```

2.2 Синтаксический анализ и генерация кода стековой машины

В рамках данной реализации компилятора синтаксический анализ выполнен методом рекурсивного спуска и дополнен поддержкой логических операций. Основные изменения касаются обработки выражений и введения новых правил для логических операторов, побитовых операций и сравнения. Новая структура анализа выражений позволяет эффективно генерировать код для более сложных логических конструкций.

2.2.1 Логические операторы

1. **Функция `logicalOrExpr`** реализует обработку логического оператора "или"(`||`).

Метод вызывает `logicalAndExpr()` и далее, пока встречается оператор логического ИЛИ, генерирует код, реализующий ленивое вычисление: если выражение истинно, дальнейшая проверка не требуется. Используются инструкции `DUP`, `JUMP_YES`, `PUSH`, `COMPARE`, что обеспечивает корректную работу логики короткого замыкания.

Вход: Последовательность логических выражений.

Выход: Сгенерированный стековый код для оператора логического "или".

2. **Функция `logicalAndExpr`** реализует обработку логического оператора "и"(`&&`).

Метод начинает с разбора побитового ИЛИ (`bitwiseOrExpr()`), затем проверяет наличие оператора `&&`. Для корректного вычисления преобразует оба операнда в логические значения (0 или 1) с помощью операций `PUSH` и `COMPARE`, после чего выполняет логическое И. Генерируется код, включающий `AND`, `DUP`, `JUMP_NO`.

Вход: Последовательность логических выражений.

Выход: Сгенерированный стековый код для оператора логического "и".

```

1 void Parser::logicalOrExpr() {
2     logicalAndExpr();
3     while (see(T_LOGICALOROP)) {

```

```

4      int dupAddress = codegen_>reserve();
5      int jumpYesAddress = codegen_>reserve();
6      next();
7      logicalAndExpr();
8      codegen_>emit(OR);
9      codegen_>emitAt(dupAddress, DUP);
10     codegen_>emitAt(jumpYesAddress, JUMP_YES, codegen_>getCurrentAddress());
11     codegen_>emit(PUSH, 0);
12     codegen_>emit(COMPARE, C_NE);
13 }
14 }
15
16 void Parser::logicalAndExpr()
17 {
18     bitwiseOrExpr();
19     while (see(T_LOGICALANDOP)) {
20         // Так как побитовое И может дать 0 даже если два операнда ненулевые
21         // (в отличии от ИЛИ, которое дает ноль только когда все биты равны нулю)
22         // нам необходимо привести каждое из чисел-операндов в 0 или 1.
23         codegen_>emit(PUSH, 0);
24         codegen_>emit(COMPARE, C_NE);
25         int dupAddress = codegen_>reserve();
26         int jumpNoAddress = codegen_>reserve();
27         next();
28         bitwiseOrExpr();
29         codegen_>emit(PUSH, 0);
30         codegen_>emit(COMPARE, C_NE);
31         codegen_>emit(AND);
32         codegen_>emitAt(dupAddress, DUP);
33         codegen_>emitAt(jumpNoAddress, JUMP_NO, codegen_>getCurrentAddress());
34     }
35 }

```

2.2.2 Побитовые операции

1. Функция **bitwiseOrExpr** обрабатывает побитовую операцию ИЛИ (|).

Вызов начинается с побитового И (**bitwiseAndExpr()**), затем производится последовательная генерация инструкции OR при встрече соответствующего токена. Операция применяется напрямую к числовым значениям, без логического приведения.

Вход: Последовательность выражений с побитовой логикой.

Выход: Сгенерированный код побитового ИЛИ.

2. Функция **bitwiseAndExpr** реализует побитовое И (&).

Работа функции основана на последовательной проверке токенов побитового И и генерации инструкции AND. Вызов **equalityExpr()** обеспечивает корректную обработку выражений с приоритетом выше.

Вход: Последовательность выражений с побитовой логикой.

Выход: Сгенерированный код побитового И.

```
1 void Parser::bitwiseOrExpr()
2 {
3     bitwiseAndExpr();
4     while (see(T_BITWISEOROP)) {
5         next();
6         bitwiseAndExpr();
7         codegen_>emit(OR);
8     }
9 }
10
11 void Parser::bitwiseAndExpr()
12 {
13     equalityExpr();
14     while (see(T_BITWISEANDOP)) {
15         next();
16         equalityExpr();
17         codegen_>emit(AND);
18     }
19 }
```

2.2.3 Изменения в обработке условий и выражений

Функция `equalityExpr` реализует разбор операций сравнения: равенства (`==`) и неравенства (`!=`).

Сначала вызывается `relation()` для обработки левой части выражения. При обнаружении оператора сравнения (`T_CMP`) метод извлекает его тип (`C_EQ`, `C_NE` и др.), после чего вызывает `relation()` для правой части. Генерируется инструкция `COMPARE` с нужным типом сравнения.

Вход: Два подвыражения и оператор сравнения.

Выход: Код сравнения для стековой машины.

```
1 void Parser::equalityExpr() {
2     relation();
3     if (see(T_CMP)) {
4         Cmp cmp = scanner_>getCmpValue();
5         if (cmp == C_EQ || cmp == C_NE) {
6             next();
7             relation();
8             codegen_>emit(COMPARE, cmp);
9         }
10    }
11 }
```

2.2.4 Модификация синтаксического анализа выражений

Функция `expression` — точка входа в синтаксический анализ выражений.

Изменены правила разбора выражений: теперь они поддерживают более сложные логические и арифметические операции. Вместо простых арифметических выражений обработка ведется с учетом приоритетов операций через отдельные функции для логических и арифметических операций

Вход: Общее выражение, включающее логические и арифметические части.

Выход: Сгенерированный стековый код для выражения с учётом приоритетов.

```
1 void Parser::expression() {
2     logicalOrExpr();
3     while (see(T_ADDOP)) {
4         Arithmetic op = scanner_->getArithmeticValue();
5         next();
6         logicalOrExpr();
7         if (op == A_PLUS) {
8             codegen_->emit(ADD);
9         } else {
10            codegen_->emit(SUB);
11        }
12    }
13 }
```

3 Результаты работы программы

На рисунках 2-123 представлены примеры программ, их р-код и вывод после их запуска

```
1 BEGIN
2
3 a := 10;
4 b := 20;
5 c := 30;
6 d := 40;
7
8
9 write(1 * (( (a) = (b) ) = 0));
10 write(2 * (( (a * b) + c ) = 230));
11 write(3 * (( a * b + c ) = 230));
12 write(4 * (( a*(b+c) ) = 500));
13 write(5 * (( (a+b) > c ) = 0));
14 write(6 * (( a + b > c ) = 0));
15 write(7 * (( a+(b>c) ) = 10));
16 write(8 * (( a + b > c ) = 0));
17 write(9 * (( (a > b) = (b > c) ) = 1));
18 write(10 * (( a > b = b > c ) = 1));
19 write(11 * (( (a = b) > (b = c) ) = 0));
20 write(12 * (( (a != b) && (c <= d) ) = 1));
21 write(13 * (( a != b && c <= d ) = 1));
22 write(14 * (( (a > b) && (c || d) ) = 0));
23 write(15 * (( a > b && c || d ) = 1));
24 write(16 * (( (a = b) && c ) = 0));
25 write(17 * (( a = b && c ) = 0));
26 write(18 * (( !(a+b) || !(b+1 | 1 + c) ) = 0));
27 write(19 * (( (a - (b | d) | b & c | d | c + d & a) ) = -2));
28 write(20 * (( (true & true) ) = 1));
29 write(21 * (( (!(a <= b) || c > 4) ) = 1));
30 write(22 * (( (!a | 8 || b + 5) ) = 1));
31 write(23 * (( !(a && b) && (!1 || b) && (!b || b) ) = 0))
32
33 /* ожидаемый вывод: числа от 1 до 23 */
34
35 END
```

Рис. 3: Пример правильно работающей программы

```
...
336:    COMPARE      0
337:    PUSH         0
338:    COMPARE      1
339:    DUP
340:    JUMP_NO      353
341:    PUSH         1
342:    PUSH         0
343:    COMPARE      0
344:    DUP
345:    JUMP_YES      348
346:    LOAD         1
347:    OR
348:    PUSH         0
349:    COMPARE      1
350:    PUSH         0
351:    COMPARE      1
352:    AND
353:    PUSH         0
354:    COMPARE      1
355:    DUP
356:    JUMP_NO      369
357:    LOAD         1
358:    PUSH         0
359:    COMPARE      0
360:    DUP
361:    JUMP_YES      364
362:    LOAD         1
363:    OR
364:    PUSH         0
365:    COMPARE      1
366:    PUSH         0
367:    COMPARE      1
368:    AND
369:    PUSH         0
370:    COMPARE      0
371:    MULT
372:    PRINT
373:    STOP
```

Рис. 4: Фрагмент р-кода этой программы

```
1 Reading input from stdin
2 1
3 2
4 3
5 4
6 5
7 6
8 7
9 8
10 9
11 10
12 11
```

```
11 12
12 13
13 14
14 15
15 16
16 17
17 18
18 19
19 20
20 21
21 22
22 23
```

Рис. 5: Вывод работы программы

```
1 begin
2
3 write(1 != 2 = 2)
```

Рис. 6: Пример программы 1 с ошибкой

```
1 begin
2
3 write(1 < 2 > 2)
```

Рис. 7: Пример программы 2 с ошибкой

Заключение

В рамках данной работы был разработан компилятор, синтаксический анализатор которого реализован методом рекурсивного спуска. Основное внимание было уделено расширению возможностей анализатора за счёт поддержки сложных выражений, включающих логические операции (AND, OR) и побитовые операции (&, |).

Достоинства

- **Short-circuit evaluation.** Для логических операторов «и» (&&) и «или» (||) была реализована оптимизация короткого замыкания:
 - при использовании оператора || — если первый операнд истинен, второй не вычисляется;
 - при использовании оператора && — если первый операнд ложен, второй не вычисляется.

Данная техника повышает производительность и обеспечивает корректную семантику логических выражений.

Недостатки

- **Повышенная сложность анализа.** Добавление логических и побитовых операций увеличивает количество правил синтаксического анализа, что приводит к росту времени компиляции.

Масштабирование

- добавление поддержки новых логических и побитовых операций (например, XOR, NOT);
- внедрение дополнительных структур управления (условные операторы, циклы);

Список источников

1. Аллахвердиев, Э.Ф. Компилятор SMilan [Электронный ресурс]. – URL: <https://tema.spbstu.ru/user-compilers/smilan.pdf> (дата обращения: 15.09.2024).
2. Карпов, Ю.Г. Теория и технология программирования. Основы построения трансляторов. – СПб.: БХВ-Петербург, 2005. – С. 272.